The Automated Supermarket Ecosystem

Comprehensive Technical Specification

Project Codename: ASE-PRIME

Document Metadata

Version: 2.0.0 (Extended Architecture)
Status: Approved for Development
Date: November 2025
Scope: End-to-End System Architecture

Prepared for the Principal Software Architect

Contents

1 Executive Summary & System Philosophy **3**
1.1 1.1 The Market Problem: Retail Friction . . . 3
1.2 1.2 The Solution: The Automated Supermarket Ecosystem (ASE) . . . 3
1.2.1 Core Design Principles . . . 3

2 Hardware Architecture: The Physical Layer **4**
2.1 2.1 The Smart Trolley (ASE-T1) . . . 4
2.1.1 Compute Unit . . . 4
2.1.2 Sensor Array . . . 4
2.1.3 Power & Actuation . . . 4
2.2 2.2 In-Store IoT Infrastructure . . . 5

3 Network Architecture: Connectivity & Cloud **6**
3.1 3.1 Network Topology . . . 6
3.1.1 The Local Store Network (Edge) . . . 6
3.1.2 The Cloud Network (Central) . . . 6

4 Service A: The Product Core (Master Data) **7**
4.1 4.1 Overview . . . 7
4.2 4.2 Technology Stack . . . 7
4.3 4.3 Key Algorithms . . . 7
4.3.1 Optimistic Concurrency Control . . . 7
4.3.2 Dynamic Pricing Engine . . . 7

5 Service B: Vision & Sensor Fusion **9**
5.1 5.1 Overview . . . 9
5.2 5.2 Technology Stack . . . 9
5.3 5.3 Computer Vision Pipeline . . . 9
5.4 5.4 Sensor Fusion Logic . . . 9

6 Service C: Fintech Vault (Blockchain) **11**
6.1 6.1 Overview . . . 11
6.2 6.2 Technology Stack . . . 11
6.3 6.3 Tokenomics: The $SPMT Token . . . 11
6.4 6.4 The ”Limited Overdraft” Smart Contract . . . 11

7 Service D: Ambient Intelligence (AI) **13**
7.1 7.1 Overview . . . 13
7.2 7.2 Technology Stack . . . 13
7.3 7.3 Recommendation Algorithms . . . 13
7.3.1 Collaborative Filtering (Item-Item) . . . 13
7.3.2 Contextual Bandits . . . 13

7.4 7.4 Privacy: Federated Learning 13

8 Service E: Autonomous Robotics **14**
8.1 8.1 Overview 14
8.2 8.2 Technology Stack 14
8.3 8.3 The Navigation Stack 14
8.4 8.4 Pathfinding Logic: A* vs. D* 14

9 Service F: The Mobile Experience **15**
9.1 9.1 Overview 15
9.2 9.2 Technology Stack 15
9.3 9.3 Offline-First Architecture 15

10 Security, Privacy & Compliance **16**
10.1 10.1 Zero Trust Security Model 16
10.2 10.2 Physical Security 16
10.3 10.3 GDPR & Data Privacy 16

11 Deployment, DevOps & Reliability **17**
11.1 11.1 CI/CD Pipeline 17
11.2 11.2 Monitoring & Observability 17
11.3 11.3 Disaster Recovery 17

Chapter 1

Executive Summary & System Philosophy

1.1 1.1 The Market Problem: Retail Friction

The traditional retail model operates on a ”Select-Wait-Pay” cycle. While the selection phase adds value, the ”Wait” and ”Pay” phases are purely friction. Studies indicate that the average consumer spends approximately 60 hours per year waiting in checkout lines. Furthermore, traditional supermarkets suffer from:

- **Inventory Latency:** Stock levels are only reconciled after sales, leading to ”ghost inventory” where systems believe items exist that have been stolen or misplaced.
- **Data Silos:** Customer behavior data is often disconnected from inventory and supply chain data.
- **High Labor Costs:** A significant percentage of operational expenditure is dedicated to cashier staffing and manual stock checking.

1.2 1.2 The Solution: The Automated Supermarket Ecosystem (ASE)

ASE eliminates the concept of a ”transaction event.” In ASE, commerce is a continuous state. The system leverages a convergence of Edge Computing, Computer Vision, and Distributed Ledger Technology (DLT) to create a ”Trustless, Frictionless, and Autonomous” environment.

1.2.1 Core Design Principles

1. **Invisible Payments:** The act of paying should be as invisible as the act of breathing. It happens automatically in the background.
2. **Decentralized Sovereignty:** Transaction data belongs to the user, secured on a blockchain, ensuring transparency and preventing centralized data manipulation.
3. **Hardware-Software Symbiosis:** The software is not just a cloud app; it is deeply integrated with custom robotics (the Smart Trolley) and IoT infrastructure.
4. **Zero-Trust Security:** No component trusts another implicitly. Every API call is authenticated; every transaction is cryptographically signed.

Chapter 2

Hardware Architecture: The Physical Layer

The ASE is not merely software; it relies on a bespoke hardware ecosystem. This chapter details the physical specifications required to support the software services.

2.1 2.1 The Smart Trolley (ASE-T1)

The Smart Trolley is the primary edge device in the ecosystem. It is a mobile computer disguised as a shopping cart.

2.1.1 Compute Unit

- **Primary Processor:** NVIDIA Jetson Orin Nano (8GB). This provides the necessary CUDA cores for real-time object detection (YOLO) and SLAM (Simultaneous Localization and Mapping) processing.
- **Secondary Controller:** ESP32 Microcontroller for low-level sensor data aggregation (weight sensors) and power management.

2.1.2 Sensor Array

- **Vision:** 2x Wide-Angle 8MP Cameras (Internal View) to cover the entire volume of the cart basket. 1x Depth Camera (Front-Facing) for collision avoidance and navigation.
- **Weight:** 4x Load Cells (one at each wheel base) integrated into a Wheatstone bridge configuration. This allows for differential weighing with a precision of ± 5 grams.
- **LiDAR:** 1x 2D LiDAR scanner (360-degree FOV) mounted on the chassis for localization and mapping.
- **IMU:** 9-Axis Inertial Measurement Unit (Accelerometer, Gyroscope, Magnetometer) to detect motion, sudden stops, or tampering.

2.1.3 Power & Actuation

- **Battery:** LiFePO4 battery pack (24V, 20Ah) providing 12 hours of continuous operation.
- **Drive System:** 2x Brushless DC Hub Motors with integrated encoders for odometry.
- **Charging:** Wireless inductive charging pads located at the ”Trolley Bay.”

2.2 2.2 In-Store IoT Infrastructure

- **Smart Shelves:** Select high-value aisles are equipped with capacitive sensing mats to detect item removal, providing a secondary data point to the Vision system.
- **BLE Beacons:** Bluetooth Low Energy beacons are placed every 5 meters in a grid pattern to facilitate indoor positioning (IPS) for the customer mobile app.
- **Edge Gateway Servers:** A local rack containing Kubernetes worker nodes to handle data offloaded from trolleys that requires higher processing power than the Jetson Orin can provide.

Chapter 3

Network Architecture: Connectivity & Cloud

3.1 3.1 Network Topology

The ASE relies on a hybrid Cloud-Edge topology to ensure low latency for robotics while maintaining high availability for financial transactions.

3.1.1 The Local Store Network (Edge)

- **Protocol:** WiFi 6E (802.11ax). This provides the bandwidth required for streaming video data from 500+ active trolleys simultaneously.
- **VLAN Segmentation:**
  - **VLAN 10 (Robotics):** Priority traffic for LiDAR data and navigation commands.
  - **VLAN 20 (Vision):** High-bandwidth video stream data.
  - **VLAN 30 (Guest):** Restricted internet access for Customer Mobile Apps.
- **Local Message Broker:** An MQTT Broker (Mosquitto) runs locally to handle telemetry data from trolleys.

3.1.2 The Cloud Network (Central)

- **Cluster Orchestration:** Kubernetes (EKS/GKE) manages the microservices (A, C, D).
- **API Gateway:** A centralized gateway (Kong or NGINX) manages ingress traffic, rate limiting, and SSL termination.
- **Inter-Service Mesh:** Istio is used to manage service-to-service communication, providing mutual TLS (mTLS) for security and advanced traffic splitting for canary deployments.

Chapter 4

Service A: The Product Core (Master Data)

4.1 4.1 Overview

Service A is the ”Source of Truth.” It maintains the master catalog of products, real-time inventory levels, and pricing logic. It is the most traffic-heavy service, receiving read requests from every active app and write requests from every active trolley.

4.2 4.2 Technology Stack

- **Language:** Node.js (TypeScript). The non-blocking event loop is ideal for the I/O-bound nature of inventory updates.
- **Database:**
  - **Primary (Write):** PostgreSQL. Used for structured product data (SKUs, Prices, Supplier Info) where relational integrity is paramount.
  - **Secondary (Read):** MongoDB. Used for unstructured metadata and flexible product attributes (e.g., ”Gluten-Free,” ”Local Origin”).
- **Caching:** Redis Cluster. Stores the ”Hot Inventory” count. When an item is picked, the count decrements in Redis instantly; the write-back to PostgreSQL happens asynchronously to ensure zero latency at the edge.

4.3 4.3 Key Algorithms

4.3.1 Optimistic Concurrency Control

To prevent ”race conditions” where two users pick the last item simultaneously:

1. User A picks Item X. System reads Version v1 of stock.
2. System attempts to update stock to (Current - 1) WHERE version = v1.
3. If the update fails (because User B updated it to v2), the system retries with the new value.

4.3.2 Dynamic Pricing Engine

Service A includes a rules engine that can adjust pricing based on expiration dates.

- *Logic:* IF (Days_To_Expiry < 2) AND (Stock_Level > Threshold) THEN Apply_Discount(20%).
- This information is pushed to the Electronic Shelf Labels (ESL) and the Mobile App via Web-Sockets.

Chapter 5

Service B: Vision & Sensor Fusion

5.1 5.1 Overview

Service B acts as the ”Sensory Cortex.” It runs primarily on the Edge (the Smart Trolley) to reduce bandwidth, with a fallback to the Edge Gateway for complex analysis.

5.2 5.2 Technology Stack

- **Language:** Python (for Model Training) and C++ (for Inference).
- **Frameworks:** PyTorch for training; NVIDIA TensorRT for optimizing models to run on the Jetson Nano.
- **Libraries:** OpenCV for image pre-processing; DeepStream SDK for video pipeline management.

5.3 5.3 Computer Vision Pipeline

1. **Trigger:** The Load Cell detects a weight stabilization event ($dWeight > 10g$).
2. **Capture:** The internal camera grabs a burst of 5 frames.
3. **Pre-processing:** Frames are cropped to the region of interest (ROI) and normalized.
4. **Inference (YOLOv8):** The frames are passed through a YOLOv8 Nano model trained on the supermarket’s 50,000 SKU dataset.
5. **Output:** The model returns a Class ID (e.g., ”734-Banana”) and a Confidence Score (e.g., 0.96).

5.4 5.4 Sensor Fusion Logic

To achieve industrial-grade reliability, Service B employs a Kalman Filter or a Bayesian Network to fuse data:

- **Visual Input:** Class=”Cereal Box”, Confidence=0.85.
- **Weight Input:** Delta=450g.

- **Database Lookup:** Service A confirms ”Cereal Box” standard weight is 450g $\pm$ 10g.
- **Result:** Fusion Probability > 0.99. **Action: ADD_TO_CART.**

Conflict Resolution: If Visual=”Cereal” but Weight=”1000g” (Brick), the system rejects the addition and flashes a red LED on the trolley.

Chapter 6

Service C: Fintech Vault (Blockchain)

6.1 6.1 Overview

Service C manages the ecosystem's economy. It is responsible for the $SPMT token, user wallets, and the immutable recording of all sales.

6.2 6.2 Technology Stack

- **Blockchain Framework:** Hyperledger Besu or Quorum (Private Ethereum-based chains). We choose an EVM-compatible private chain to allow for high throughput (1000+ TPS) and zero gas fees for end-users.
- **Consensus Mechanism:** IBFT 2.0 (Istanbul Byzantine Fault Tolerance). This ensures immediate block finality, which is crucial for retail (we cannot wait for probabilistic finality like Bitcoin).
- **Smart Contract Language:** Solidity.

6.3 6.3 Tokenomics: The $SPMT Token

The ecosystem utilizes a dual-token structure managed internally, though the user only sees $SPMT.

1. **$SPMT (Stablecoin):** Pegged 1:1 with the local fiat currency. Used for all purchases.
2. **$ASE-GOV (Governance):** A token earned by loyal customers that grants voting rights on store policies (e.g., which new products to stock).

6.4 6.4 The "Limited Overdraft" Smart Contract

A unique feature of the ASE DeFi layer is the trusted credit protocol.

- **Logic:**

```
function checkout(address user, uint amount) public {
    uint balance = balanceOf(user);
    if (balance >= amount) {
        _transfer(user, storeWallet, amount);
```

```
        } else if (isSubscriber(user) && (balance - amount >= overdraftLimit)) {
            // Allow negative balance
            _transfer(user, storeWallet, amount);
            emit OverdraftUsed(user, amount - balance);
            triggerFiatRecovery(user); // Call Service C backend
        } else {
            revert("Insufficient funds");
        }
    }
```

Chapter 7

Service D: Ambient Intelligence (AI)

7.1 7.1 Overview

Service D transforms the store from a passive warehouse into an active agent. It analyzes user behavior in real-time to provide recommendations and optimize store layout.

7.2 7.2 Technology Stack

- **Framework:** TensorFlow Extended (TFX) for production ML pipelines.
- **Vector Database:** Milvus. Used to store user profiles and product features as high-dimensional vectors.

7.3 7.3 Recommendation Algorithms

7.3.1 Collaborative Filtering (Item-Item)

"Users who bought X also bought Y." This is standard, but ASE adds real-time context.

7.3.2 Contextual Bandits

The recommendation engine is treated as a Reinforcement Learning problem.

- **State:** User's current location (Aisle 4), Current Cart (Pasta, Oil), Time (6:00 PM).
- **Action:** Recommend "Basil" vs. Recommend "Wine."
- **Reward:** User adds the item to the cart (+1) or ignores it (0).

7.4 7.4 Privacy: Federated Learning

To respect user privacy, Service D implements Federated Learning.

- The model training happens *on the user's phone* (Service F).
- Only the *model weights* (gradients) are sent back to the central server to improve the global model.
- The raw purchase history data never leaves the user's device in an identifiable format.

Chapter 8

Service E: Autonomous Robotics

8.1 8.1 Overview

Service E manages the physical movement of the Smart Trolleys. It ensures they return to the charging bay autonomously and can navigate through a crowded store without collision.

8.2 8.2 Technology Stack

- **OS:** ROS 2 (Robot Operating System) - Humble Hawksbill distribution.
- **Middleware:** DDS (Data Distribution Service) for real-time communication between robot nodes.
- **Simulation:** Gazebo. Used to test navigation logic in a virtual store before deploying to physical trolleys.

8.3 8.3 The Navigation Stack

The trolley uses a layered costmap approach for navigation.

1. **Static Layer:** The map of shelves and walls (Permanent).
2. **Obstacle Layer:** Dynamic objects detected by LiDAR (Shoppers, Boxes).
3. **Inflation Layer:** A safety buffer around obstacles that the robot treats as ”lethal” terrain.

8.4 8.4 Pathfinding Logic: A* vs. D*

- **A* (A-Star):** Used when the store is empty (Night Mode) for optimal pathing.
- **D* (Dynamic A*):** Used during operating hours. D* allows the robot to re-plan a path efficiently when a new obstacle (a person) suddenly appears in a previously free path.

Chapter 9

Service F: The Mobile Experience

9.1 9.1 Overview

Service F is the user's remote control for the physical world. It handles authentication, wallet management, and live shopping visualization.

9.2 9.2 Technology Stack

- **Framework:** Flutter (Dart). Chosen for its ability to compile to native ARM code, ensuring high performance for map rendering and animation.
- **State Management:** BLoC (Business Logic Component). This pattern separates the UI from the business logic, crucial for handling the asynchronous streams of data coming from the WebSocket (Service A).

9.3 9.3 Offline-First Architecture

Supermarkets often have poor cellular reception.

- The app utilizes a local SQLite database (drift) to cache the product catalog.
- If the network drops, the app queues "Add to Cart" events locally.
- When connectivity is restored, the app performs a "State Reconciliation" with Service A to ensure the cart total is accurate.

Chapter 10

Security, Privacy & Compliance

10.1 10.1 Zero Trust Security Model

The ASE assumes that the local network is hostile (e.g., a hacker could plug into a store ethernet port).

- **mTLS:** Every service (Vision to Product Core, Robotics to Fleet Manager) communicates using mutual TLS. Both client and server verify certificates.
- **JWT Tokens:** All API requests must bear a short-lived JSON Web Token signed by the Auth Service.

10.2 10.2 Physical Security

- **Anti-Tamper:** The Smart Trolleys are equipped with micro-switches. If the casing is opened to access the Jetson Nano, the device instantly wipes its encryption keys and enters a ”Bricked” state.
- **Geofencing:** If a trolley is taken outside the store parking lot (detected via GPS), the wheel motors lock automatically and a beacon alert is sent to recovery teams.

10.3 10.3 GDPR & Data Privacy

- **Right to be Forgotten:** Since transactions are on an immutable blockchain, we cannot ”delete” them. However, we store the transaction linked to a *Anonymous ID*. The mapping of *Anonymous ID* to *Real User* is stored in an off-chain SQL database. To ”forget” a user, we delete the off-chain mapping, rendering the on-chain history mathematically anonymous and compliant with GDPR.

Chapter 11

Deployment, DevOps & Reliability

11.1 11.1 CI/CD Pipeline

We utilize a GitOps workflow.

- **Code Commit:** Developer pushes code to GitLab.
- **CI:** GitLab CI runs Unit Tests (Jest/PyTest) and Static Analysis (SonarQube).
- **Build:** Docker images are built and pushed to a private Registry (Harbor).
- **CD:** ArgoCD detects the new image version and updates the Kubernetes manifests.

11.2 11.2 Monitoring & Observability

- **Metrics:** Prometheus scrapes metrics from all services (e.g., ”Active Carts,” ”Transaction Latency,” ”Vision Model Confidence”).
- **Visualization:** Grafana dashboards display real-time store health.
- **Tracing:** Jaeger is used for distributed tracing. We can trace a single ”Add to Cart” request as it travels from Trolley $\rightarrow$ Gateway $\rightarrow$ Service A $\rightarrow$ Database.

11.3 11.3 Disaster Recovery

- **Database Replication:** PostgreSQL uses Write-Ahead Logging (WAL) to replicate data to a standby server in a different Availability Zone.
- **Hybrid Failover:** If the Cloud Connection is severed, the Edge Gateway in the store can run in ”Offline Mode,” caching transactions locally and syncing with the Blockchain once connectivity is restored.